Announcements

- Pset 4 update (due to solution leak):
- SRE 2 next Mon.
- Pset 4 due next Mon (in-class reflection on the Monday after).
- Anurag OH after class (SEC 3.323).
- HV 12.1-12.4, 13.1-13.2

Extended Church-Turing thesis

*Extended Church–Turing Thesis: Every **physically realizable model** of computation can be simulated by a Word-RAM program with only a polynomial slowdown in runtime. Conversely, Word-RAM programs can be physically simulated in real time only polynomially slower than their defined runtime.*

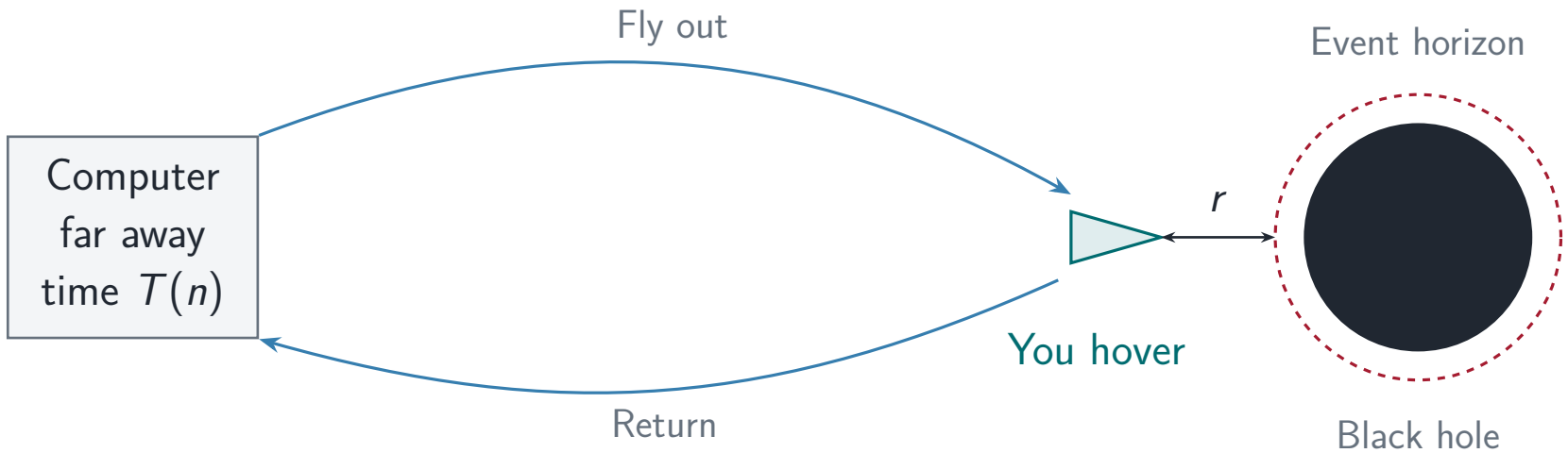
Extended Church-Turing thesis

*Extended Church–Turing Thesis: Every **physically realizable model** of computation can be simulated by a Word-RAM program with only a polynomial slowdown in runtime. Conversely, Word-RAM programs can be physically simulated in real time only polynomially slower than their defined runtime.*

Physically realizable in the entire universe!

The Interstellar Model of Computation

Every algorithm in one minute?

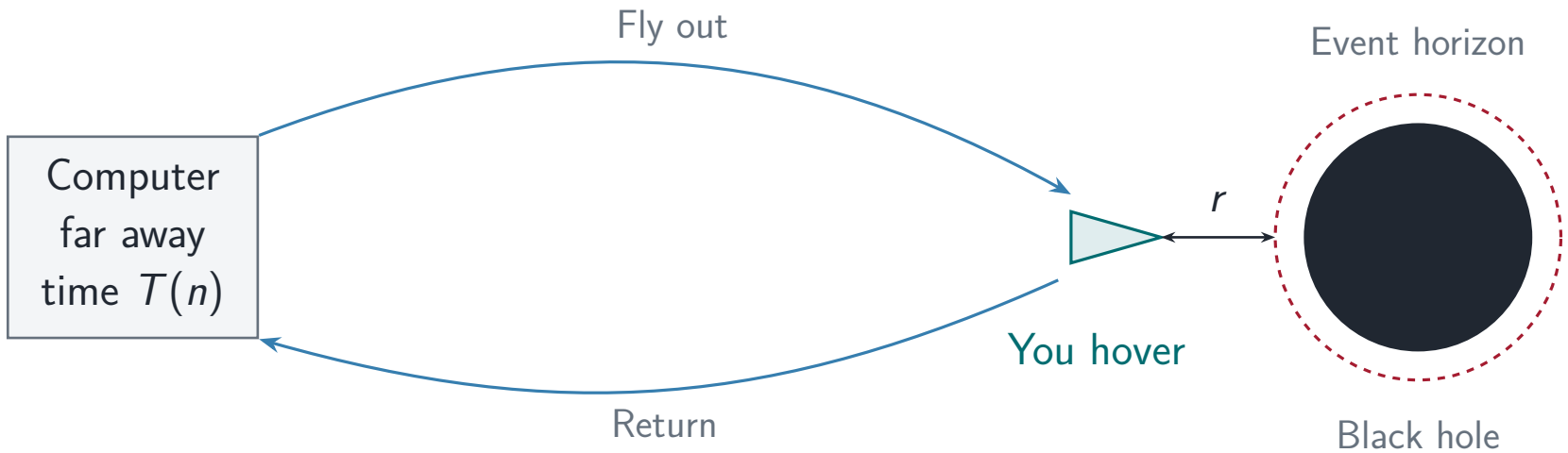


$$t_{\text{you}} = t_{\text{computer}} \sqrt{1 - \frac{1000}{r + 1000}}.$$

for a 1000 meter radius black hole.

The Interstellar Model of Computation

Every algorithm in one minute?



$$t_{\text{you}} = t_{\text{computer}} \sqrt{1 - \frac{1000}{r + 1000}}.$$

for a 1000 meter radius black hole. Since $t_{\text{you}} = 1$, $t_{\text{computer}} = T(n)$, $r = \frac{1000}{T^2(n)-1}$.

The Interstellar Model of Computation

- Computer may run either Exhaustive search sort ($T_{exh}(n) = O(n \cdot n!)$) or Merge sort ($T_{merge}(n) = O(n \log n)$). For you, both finish in 1 sec!

The Interstellar Model of Computation

- Computer may run either Exhaustive search sort ($T_{exh}(n) = O(n \cdot n!)$) or Merge sort ($T_{merge}(n) = O(n \log n)$). For you, both finish in 1 sec!
- Resolution: to hover near black hole, you need to fight its strong gravity. So you need lots of fuel. Refueling time scales with $T(n)$.

The Interstellar Model of Computation

- Computer may run either Exhaustive search sort ($T_{exh}(n) = O(n \cdot n!)$) or Merge sort ($T_{merge}(n) = O(n \log n)$). For you, both finish in 1 sec!
- Resolution: to hover near black hole, you need to fight its strong gravity. So you need lots of fuel. Refueling time scales with $T(n)$.
  In other words, the command
  refuel(r)
  needs to be included in this model of computation.

Graphs: definitions

A *directed graph* (or digraph) $G = (V, E)$ consists of:

- A finite set $V = V(G)$ of *vertices*.
- A set $E = E(G)$ of *edges*: ordered pairs (u, v) with $u, v \in V$ and $u \neq v$.

Graphs: definitions

A *directed graph* (or digraph) $G = (V, E)$ consists of:

- A finite set $V = V(G)$ of *vertices*.
- A set $E = E(G)$ of *edges*: ordered pairs (u, v) with $u, v \in V$ and $u \neq v$.

$$n = |V|, \qquad m = |E|.$$

Undirected graphs: unordered pairs $\{u, v\}$.

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Q: How do we model a road network?

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Q: How do we model a road network?

- Vertices represent intersections; edges represent road segments.

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Q: How do we model a road network?

- Vertices represent intersections; edges represent road segments.

Variants of the model

- **Weighted:** each edge has a weight for travel time (such as graveled roads take longer).

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Q: How do we model a road network?

- Vertices represent intersections; edges represent road segments.

Variants of the model

- **Weighted:** each edge has a weight for travel time (such as graveled roads take longer).
- **Multigraph:** allow parallel edges, and possibly self-loops.

Graphs represent many settings

Motivating problem1: route finding

Given a road network, a starting point s, and a destination t, what is the shortest way to get from s to t?

Q: How do we model a road network?

- Vertices represent intersections; edges represent road segments.

Variants of the model

- **Weighted:** each edge has a weight for travel time (such as graveled roads take longer).
- **Multigraph:** allow parallel edges, and possibly self-loops.
- **Planar:** can be drawn in the plane without edge crossings (no flyovers).

Graphs represent many settings

Motivating problem2: register allocation

Graphs represent many settings

Motivating problem2: register allocation

- In RAM lecture, we saw that at most 8 registers suffice to simulate any RAM program. In special settings, we can do much better!

Graphs represent many settings

Motivating problem2: register allocation

- In RAM lecture, we saw that at most 8 registers suffice to simulate any RAM program. In special settings, we can do much better!
- Exploit the fact that most of the registers generated by compilers are *temporary* registers, whose value only needs to be maintained for a short time.

Graphs represent many settings

Motivating problem2: register allocation

- In RAM lecture, we saw that at most 8 registers suffice to simulate any RAM program. In special settings, we can do much better!
- Exploit the fact that most of the registers generated by compilers are *temporary* registers, whose value only needs to be maintained for a short time.
- Define a live region for each register, and use a graph to represent conflicts.

A Word-RAM program

```
   SquaredDistanceToNegativeOne(x):
   Input          : An array x = (x[0], x[1], ..., x[n − 1])
   Output         : (x[0] + 1)^2 + (x[1] + 1)^2 + ··· + (x[n − 1] + 1)^2
   Variables      : input_len, output_len, output_ptr, temp_0, temp_1, temp_2, temp_3
 0 output_ptr = input_len;
 1 output_len = 1;
 2 temp_3 = 0;
 3     IF input_len == 0 GOTO 15;
 4     temp_0 = 1;
 5     temp_0 = temp_0 + temp_3;
 6     input_len = input_len − temp_0;
 7     temp_1 = M[input_len];
 8     temp_1 = temp_1 + temp_0;
 9     temp_1 = temp_1 × temp_1;
10     temp_2 = M[output_ptr];
11     temp_2 = temp_2 + temp_1;
12     temp_3 = 0;
13     M[output_ptr] = temp_2;
14     IF temp_3 == 0 GOTO 3;
15 HALT ;                                   /* not an actual command */
```

Live regions

The live regions for the temporary variables $\mathrm{temp}_0, \mathrm{temp}_1, \mathrm{temp}_2, \mathrm{temp}_3$ are:

Live regions

The live regions for the temporary variables $\mathrm{temp}_0, \mathrm{temp}_1, \mathrm{temp}_2, \mathrm{temp}_3$ are:

$$
\begin{aligned}
R_0 &= \{4, 5, 6, 7, 8\} \\
R_1 &= \{7, 8, 9, 10, 11\} \\
R_2 &= \{10, 11, 12, 13\} \\
R_3 &= \{2, 3, 4, 5, 12, 13, 14\}
\end{aligned}
$$

Live regions

The live regions for the temporary variables $\mathrm{temp}_0, \mathrm{temp}_1, \mathrm{temp}_2, \mathrm{temp}_3$ are:

$$
\begin{aligned}
R_0 &= \{4, 5, 6, 7, 8\} \\
R_1 &= \{7, 8, 9, 10, 11\} \\
R_2 &= \{10, 11, 12, 13\} \\
R_3 &= \{2, 3, 4, 5, 12, 13, 14\}
\end{aligned}
$$

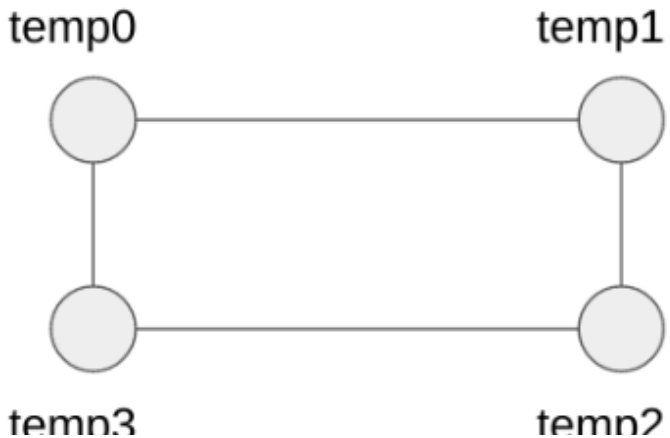
Conflict graph (vertices - registers, edges between vertices if the live regions overlap). Note that this is undirected.

Live regions

The live regions for the temporary variables $\text{temp}_0, \text{temp}_1, \text{temp}_2, \text{temp}_3$ are:

$$
\begin{aligned}
R_0 &= \{4, 5, 6, 7, 8\} \\
R_1 &= \{7, 8, 9, 10, 11\} \\
R_2 &= \{10, 11, 12, 13\} \\
R_3 &= \{2, 3, 4, 5, 12, 13, 14\}
\end{aligned}
$$

Conflict graph (vertices - registers, edges between vertices if the live regions overlap). Note that this is undirected.



Representing Graphs in our model of computation

Adjacency list representation of digraph:

- Label vertices $0, \ldots, n-1$. Use the Word-RAM with $w = \Theta(\log n)$, so a vertex name fits in one word.
- For each vertex v, the array $\text{Nbr}_{\text{out}}[v]$ of its out-neighbors is:

Representing Graphs in our model of computation

Adjacency list representation of digraph:

- Label vertices $0, \ldots, n-1$. Use the Word-RAM with $w = \Theta(\log n)$, so a vertex name fits in one word.
- For each vertex v, the array $\mathrm{Nbr}_{\mathrm{out}}[v]$ of its out-neighbors is:

$$N_{\mathrm{out}}(v) = \{u : (v, u) \in E\}, \qquad \mathrm{d}_{\mathrm{out}}(v) = |N_{\mathrm{out}}(v)|.$$

- The representation lists $\mathrm{Nbr}_{\mathrm{out}}[v]$ and $\mathrm{d}_{\mathrm{out}}(v)$ for each v.

Representing Graphs in our model of computation

Adjacency list representation of digraph:

- Label vertices $0, \ldots, n-1$. Use the Word-RAM with $w = \Theta(\log n)$, so a vertex name fits in one word.
- For each vertex v, the array $\mathrm{Nbr}_{\mathrm{out}}[v]$ of its out-neighbors is:

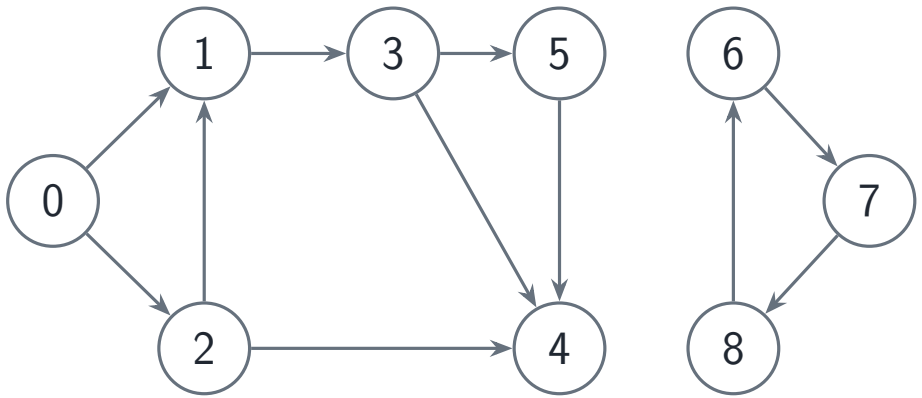
$$N_{\mathrm{out}}(v) = \{u : (v, u) \in E\}, \qquad \mathrm{d}_{\mathrm{out}}(v) = |N_{\mathrm{out}}(v)|.$$

- The representation lists $\mathrm{Nbr}_{\mathrm{out}}[v]$ and $\mathrm{d}_{\mathrm{out}}(v)$ for each v.

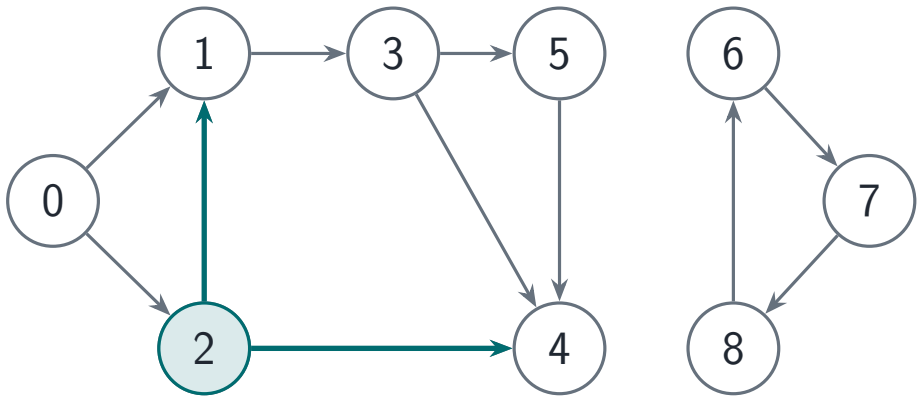
Enumerating the out-neighbors of v takes $O(1 + \mathrm{d}_{\mathrm{out}}(v))$ time.

Undirected graphs: For each vertex v, the array $\mathrm{Nbr}(v)$ is the list of neighbors of v.

Representing Graphs



Representing Graphs



v	$\text{Nbr}_{\text{out}}[v]$
0	[1, 2]
1	[3]
2	[1, 4]
3	[4, 5]
4	[ ]
5	[4]
6	[7]
7	[8]
8	[6]

$N_{\text{out}}(2) = \{1, 4\}$ and $\text{d}_{\text{out}}(2) = 2$.

Shortest Walks

Problem: ShortestWalk

- **Input:** a digraph $G = (V, E)$ and vertices $s, t \in V$.
- **Output:** a shortest walk from s to t, if one exists.

Shortest Walks

Problem: ShortestWalk

- **Input:** a digraph $G = (V, E)$ and vertices $s, t \in V$.
- **Output:** a shortest walk from s to t, if one exists.

Definition: walk A sequence $v_0, v_1, \ldots, v_\ell$ such that

$$v_0 = s, \qquad v_\ell = t, \qquad (v_{i-1}, v_i) \in E \quad (1 \le i \le \ell).$$

Shortest Walks

Problem: ShortestWalk

- **Input:** a digraph $G = (V, E)$ and vertices $s, t \in V$.
- **Output:** a shortest walk from s to t, if one exists.

Definition: walk A sequence $v_0, v_1, \ldots, v_\ell$ such that

$$v_0 = s, \qquad v_\ell = t, \qquad (v_{i-1}, v_i) \in E \quad (1 \le i \le \ell).$$

A *path* is a walk with no repeated vertices.

Shortest Walks

Length and distance

- The length of $w = (v_0, \ldots, v_\ell)$ is its number of edges: ℓ.

Shortest Walks

Length and distance

- The length of $w = (v_0, \ldots, v_\ell)$ is its number of edges: ℓ.
- The distance from s to t is

$$\mathrm{dist}_G(s,t) = \begin{cases} \min\{\mathrm{length}(w) : w \text{ is an } s\text{-to-}t \text{ walk}\}, & \text{if one exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

Shortest Walks

Length and distance

- The length of $w = (v_0, \ldots, v_\ell)$ is its number of edges: ℓ.
- The distance from s to t is

$$\text{dist}_G(s,t) = \begin{cases} \min\{\text{length}(w) : w \text{ is an } s\text{-to-}t \text{ walk}\}, & \text{if one exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

A *shortest walk* has length $\text{dist}_G(s,t)$.

In particular, $\text{dist}_G(s,s) = 0$: take the walk consisting only of s.

Shortest Walks

Q: What algorithm is immediate from the definition, if you are allowed to take a long time?

Shortest Walks

Q: What algorithm is immediate from the definition, if you are allowed to take a long time?
Exhaustive search: enumerate walks from s in increasing length, stopping at the first one that reaches t.

Shortest Walks

Q: What algorithm is immediate from the definition, if you are allowed to take a long time?
Exhaustive search: enumerate walks from s in increasing length, stopping at the first one that reaches t.

Q: When can we stop and conclude there is no walk?

Shortest Walks

Q: What algorithm is immediate from the definition, if you are allowed to take a long time?
Exhaustive search: enumerate walks from s in increasing length, stopping at the first one that reaches t.

Q: When can we stop and conclude there is no walk?
Lemma. Every shortest walk is a path.

A finite shortest distance is at most $n - 1$.

Shortest Walks

Q: What algorithm is immediate from the definition, if you are allowed to take a long time?
Exhaustive search: enumerate walks from s in increasing length, stopping at the first one that reaches t.

Q: When can we stop and conclude there is no walk?
Lemma. Every shortest walk is a path.

A finite shortest distance is at most $n - 1$.

This means we can cap the search to talks of length at most $n - 1$. The exhaustive enumeration can still take exponential time.

Breadth-First Search

Related Problem: SingleSourceDistances

- **Input:** a digraph G and a source s.
- **Output:** an array with $\text{dist}[v] = \text{dist}_G(s, v)$ for every v.

We will first show how to solve SingleSourceDistances and then show how to solve ShortestWalk. The former will be solved using BFS.

Breadth-First Search

Related Problem: SingleSourceDistances

- **Input:** a digraph G and a source s.
- **Output:** an array with $\text{dist}[v] = \text{dist}_G(s, v)$ for every v.

We will first show how to solve SingleSourceDistances and then show how to solve ShortestWalk. The former will be solved using BFS.

- BFS explores the graph. It maintains S: all vertices discovered so far, F: the *frontier*, discovered in the most recent round.
- The next frontier consists of previously undiscovered out-neighbors of F.

Breadth-First Search

Related Problem: SingleSourceDistances

- **Input:** a digraph G and a source s.
- **Output:** an array with $\text{dist}[v] = \text{dist}_G(s, v)$ for every v.

We will first show how to solve SingleSourceDistances and then show how to solve ShortestWalk. The former will be solved using BFS.

- BFS explores the graph. It maintains S: all vertices discovered so far, F: the *frontier*, discovered in the most recent round.
- The next frontier consists of previously undiscovered out-neighbors of F.

Algorithm explores the whole graph without knowing the destination: “I don’t know where I am going, but I am on my way".

Breadth-First Search

Initialize $\text{dist}[v] \leftarrow \infty$ for every $v \in V$
$S \leftarrow \{s\}$; $F \leftarrow \{s\}$; $\text{dist}[s] \leftarrow 0$
for $d = 1, \ldots, n-1$ **do**

return dist

Breadth-First Search

```
1 Initialize dist[v] ← ∞ for every v ∈ V
2 S ← {s};   F ← {s};   dist[s] ← 0
3 for d = 1, ..., n − 1 do
4 |   F_next ← {v ∉ S : ∃u ∈ F with (u, v) ∈ E}
  |
  |
  |
8 return dist
```

Use the *old* frontier to construct the *next* frontier.

Breadth-First Search

Initialize dist$[v] \leftarrow \infty$ for every $v \in V$
$S \leftarrow \{s\}$; $F \leftarrow \{s\}$; dist$[s] \leftarrow 0$
for $d = 1, \ldots, n-1$ **do**
    $F_{\text{next}} \leftarrow \{v \notin S : \exists u \in F \text{ with } (u, v) \in E\}$
    For each $v \in F_{\text{next}}$, set dist$[v] \leftarrow d$
return dist

Use the *old* frontier to construct the *next* frontier.

Breadth-First Search

```
1 Initialize dist[v] ← ∞ for every v ∈ V
2 S ← {s};   F ← {s};   dist[s] ← 0
3 for d = 1, ..., n − 1 do
4 |   F_next ← {v ∉ S : ∃u ∈ F with (u, v) ∈ E}
5 |   For each v ∈ F_next, set dist[v] ← d
6 |   S ← S ∪ F_next
8 return dist
```

Use the *old* frontier to construct the *next* frontier.

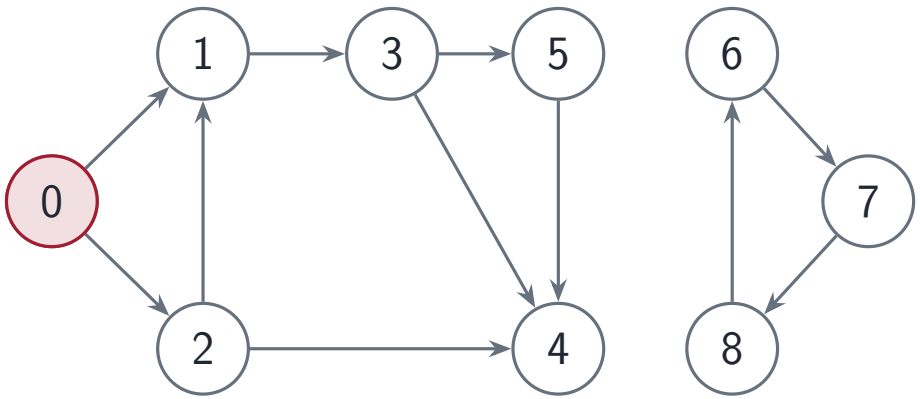
Breadth-First Search

```
1 Initialize dist[v] ← ∞ for every v ∈ V
2 S ← {s};   F ← {s};   dist[s] ← 0
3 for d = 1, ..., n − 1 do
4 |   F_next ← {v ∉ S : ∃u ∈ F with (u, v) ∈ E}
5 |   For each v ∈ F_next, set dist[v] ← d
6 |   S ← S ∪ F_next
7 |   F ← F_next
8 return dist
```

Use the *old* frontier to construct the *next* frontier.

Breadth-First Search

Example: source $s = 0$



$F_0 = \{0\}$

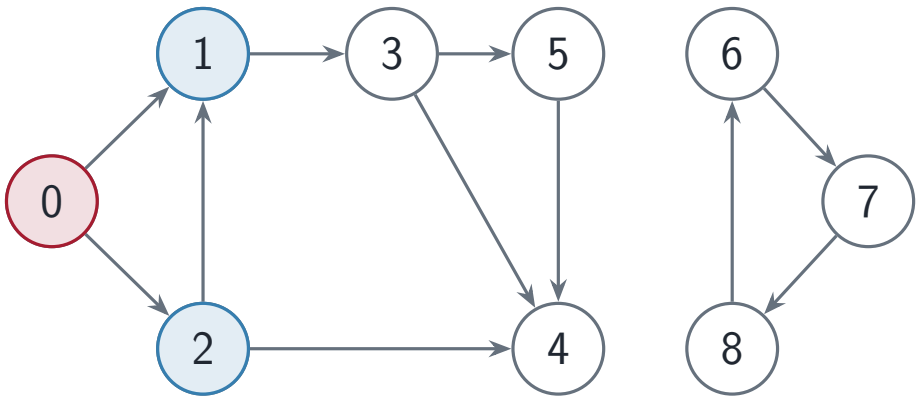
$F_1 =$

$F_2 =$

$F_3 =$

Breadth-First Search

Example: source $s = 0$



$F_0 = \{0\}$

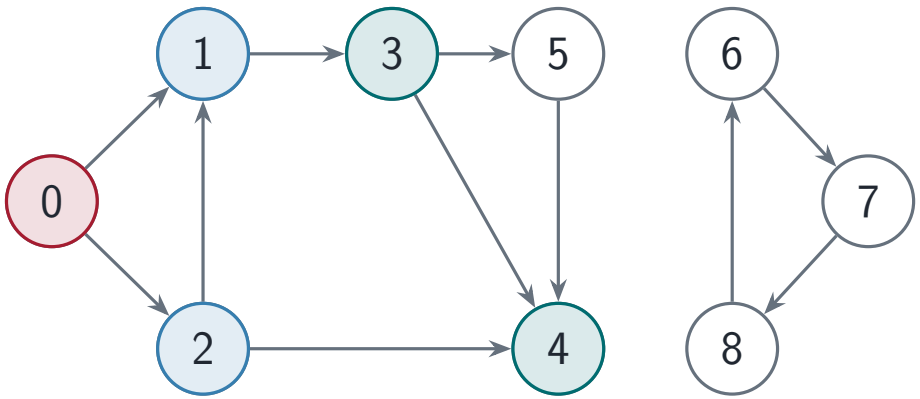
$F_1 = \{1, 2\}$

$F_2 =$

$F_3 =$

Breadth-First Search

Example: source $s = 0$



$F_0 = \{0\}$

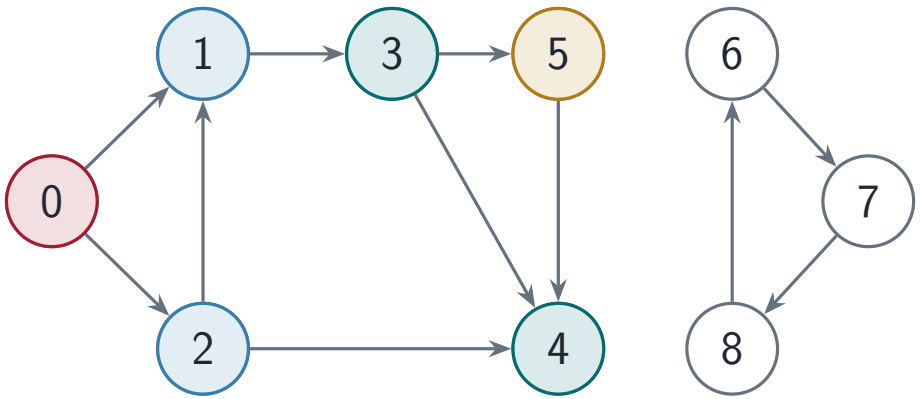
$F_1 = \{1, 2\}$

$F_2 = \{3, 4\}$

$F_3 =$

Breadth-First Search

Example: source $s = 0$



$F_0 = \{0\}$

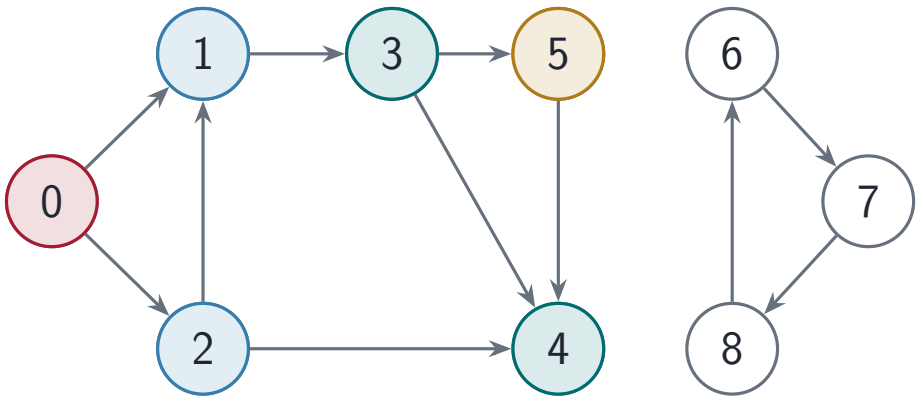
$F_1 = \{1, 2\}$

$F_2 = \{3, 4\}$

$F_3 = \{5\}$

Breadth-First Search

Example: source $s = 0$



$F_0 = \{0\}$

$F_1 = \{1, 2\}$

$F_2 = \{3, 4\}$

$F_3 = \{5\}$

Vertices $6, 7, 8$ are unreachable: their distances remain ∞.

Breadth-First Search: Correctness Proof

We will argue that BFS correctly solves SingleSourceDIstances. The proof is by induction.

Breadth-First Search: Correctness Proof

We will argue that BFS correctly solves SingleSourceDIstances. The proof is by induction. Let F_d, S_d be the sets after round d, starting with $F_0 = S_0 = \{s\}$.

Breadth-First Search: Correctness Proof

We will argue that BFS correctly solves SingleSourceDIstances. The proof is by induction.
Let F_d, S_d be the sets after round d, starting with $F_0 = S_0 = \{s\}$.
Induction hypothesis $P(d)$: The following equality holds:

$$F_d = \{v : \text{dist}_G(s, v) = d\}, \qquad S_d = \{v : \text{dist}_G(s, v) \leq d\}.$$

Breadth-First Search: Correctness Proof

We will argue that BFS correctly solves SingleSourceDIstances. The proof is by induction.
Let F_d, S_d be the sets after round d, starting with $F_0 = S_0 = \{s\}$.
Induction hypothesis $P(d)$: The following equality holds:

$$F_d = \{v : \text{dist}_G(s, v) = d\}, \qquad S_d = \{v : \text{dist}_G(s, v) \leq d\}.$$

Q: Why is this the right induction hypothesis?

Breadth-First Search: Correctness Proof

We will argue that BFS correctly solves SingleSourceDIstances. The proof is by induction.
Let F_d, S_d be the sets after round d, starting with $F_0 = S_0 = \{s\}$.
Induction hypothesis $P(d)$: The following equality holds:

$$F_d = \{v : \text{dist}_G(s, v) = d\}, \qquad S_d = \{v : \text{dist}_G(s, v) \leq d\}.$$

Q: Why is this the right induction hypothesis?
Base case: $P(0)$ holds since s is the only vertex that has distance 0 from s.

Breadth-First Search: Correctness Proof

Inductive step

- Assume $P(d-1)$ holds:

$$F_{d-1} = \{v : \text{dist}_G(s, v) = d - 1\}, \qquad S_{d-1} = \{v : \text{dist}_G(s, v) \leq d - 1\}.$$

- Now, we show $P(d)$.

Breadth-First Search: Correctness Proof

Inductive step

- Assume $P(d-1)$ holds:

$$F_{d-1} = \{v : \text{dist}_G(s, v) = d-1\}, \qquad S_{d-1} = \{v : \text{dist}_G(s, v) \leq d-1\}.$$

- Now, we show $P(d)$.
- Recall from Line 4, $F_d = \{v \notin S_{d-1}, \exists u \in F_{d-1} \text{ with } (u, v) \in E\}$. This implies $F_d = \{v : \text{dist}_G(s, v) = d\}$ (show this).

Breadth-First Search: Correctness Proof

Inductive step

- Assume $P(d-1)$ holds:

$$F_{d-1} = \{v : \text{dist}_G(s, v) = d-1\}, \qquad S_{d-1} = \{v : \text{dist}_G(s, v) \leq d-1\}.$$

- Now, we show $P(d)$.
- Recall from Line 4, $F_d = \{v \notin S_{d-1}, \exists u \in F_{d-1} \text{ with } (u, v) \in E\}$. This implies $F_d = \{v : \text{dist}_G(s, v) = d\}$ (show this).
- Line 6 sets $S_d = S_{d-1} \cup F_d$, showing that $S_d = \{v : \text{dist}_G(s, v) \leq d\}$.

Breadth-First Search: Correctness Proof

Inductive step

- Assume $P(d-1)$ holds:

$$F_{d-1} = \{v : \text{dist}_G(s, v) = d-1\}, \qquad S_{d-1} = \{v : \text{dist}_G(s, v) \leq d-1\}.$$

- Now, we show $P(d)$.
- Recall from Line 4, $F_d = \{v \notin S_{d-1}, \exists u \in F_{d-1} \text{ with } (u, v) \in E\}$. This implies $F_d = \{v : \text{dist}_G(s, v) = d\}$ (show this).
- Line 6 sets $S_d = S_{d-1} \cup F_d$, showing that $S_d = \{v : \text{dist}_G(s, v) \leq d\}$.

Therefore,

$$F_d = \{v : \text{dist}_G(s, v) = d\}, \qquad S_d = S_{d-1} \cup F_d = \{v : \text{dist}_G(s, v) \leq d\}.$$

Breadth-First Search: Runtime Analysis

Runtime cost (since each vertex is visited once and updates use the neighbors of that vertex):

$$O(n) + O\left(\sum_{u \text{ processed}} (1 + \mathrm{d}_{\mathrm{out}}(u))\right) \leq O\left(n + \sum_{u \in V} \mathrm{d}_{\mathrm{out}}(u)\right) = O(n + m).$$

Breadth-First Search: Runtime Analysis

Runtime cost (since each vertex is visited once and updates use the neighbors of that vertex):

$$O(n) + O\left(\sum_{u \text{ processed}} (1 + \mathrm{d}_{\text{out}}(u))\right) \leq O\left(n + \sum_{u \in V} \mathrm{d}_{\text{out}}(u)\right) = O(n + m).$$

However, S, F are dynamically changing objects, so the way to store them matters. If stored as an array, cost is higher. Storing them as linked list leads to the above cost.

Breadth-First Search: Runtime Analysis

Runtime cost (since each vertex is visited once and updates use the neighbors of that vertex):

$$O(n) + O\left(\sum_{u \text{ processed}} (1 + \mathrm{d}_{\text{out}}(u))\right) \leq O\left(n + \sum_{u \in V} \mathrm{d}_{\text{out}}(u)\right) = O(n + m).$$

However, S, F are dynamically changing objects, so the way to store them matters. If stored as an array, cost is higher. Storing them as linked list leads to the above cost.

Theorem. BFS correctly solves SINGLESOURCEDISTANCES in $O(n + m)$ time.

Finding the Paths

Q: How do we find a shortest path, rather than just its length?

Finding the Paths

Q: How do we find a shortest path, rather than just its length? Create a pred array (the predecessor array). When BFS first discovers v from u, set

$$\text{pred}[v] \leftarrow u.$$

Finding the Paths

Q: How do we find a shortest path, rather than just its length? Create a pred array (the predecessor array). When BFS first discovers v from u, set

$$\text{pred}[v] \leftarrow u.$$

To answer a query for t:

- If $\text{dist}[t] = \infty$, report that no path exists.

Finding the Paths

Q: How do we find a shortest path, rather than just its length? Create a pred array (the predecessor array). When BFS first discovers v from u, set

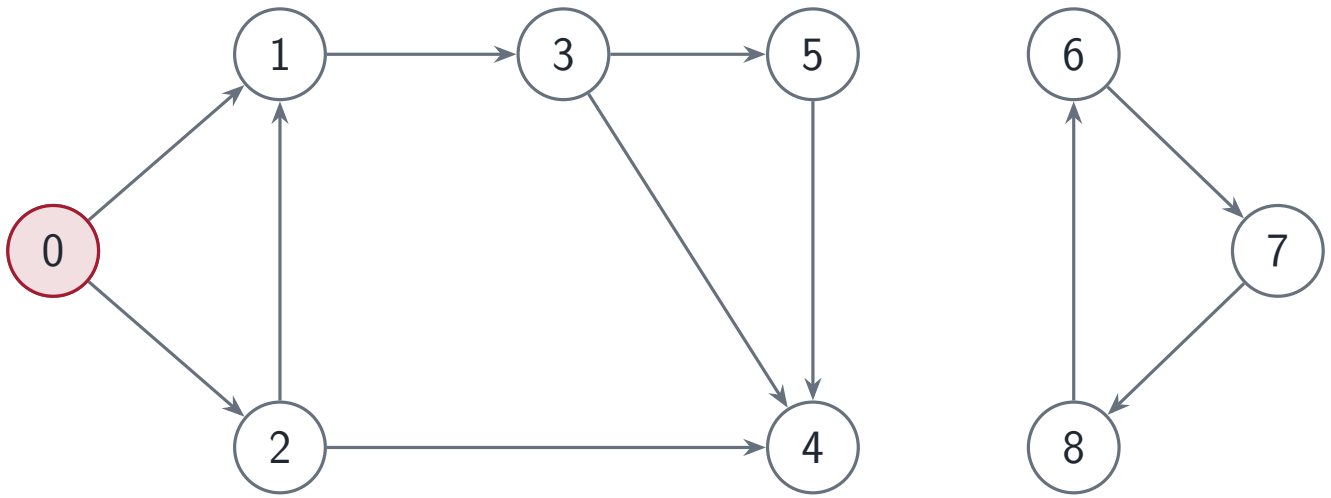
$$\text{pred}[v] \leftarrow u.$$

To answer a query for t:

- If $\text{dist}[t] = \infty$, report that no path exists.
- Otherwise, follow $t, \text{pred}[t], \text{pred}[\text{pred}[t]], \ldots$ back to s.
- Reverse this sequence to obtain the path from s to t.

Finding the Paths

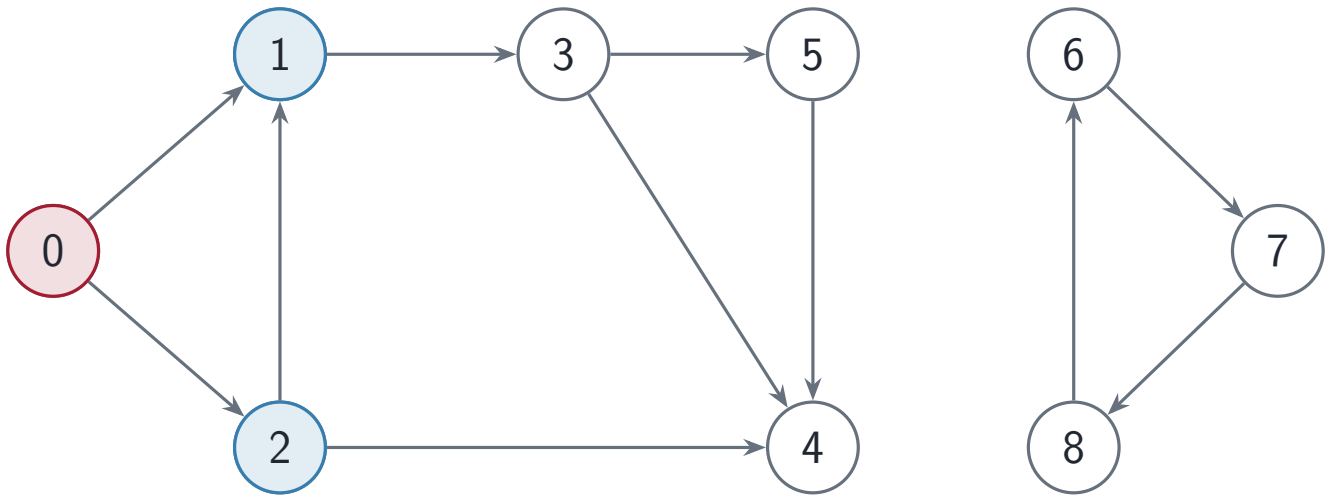
Example: $s = 0,\ t = 4$



$F_0 = \{0\}$

Finding the Paths

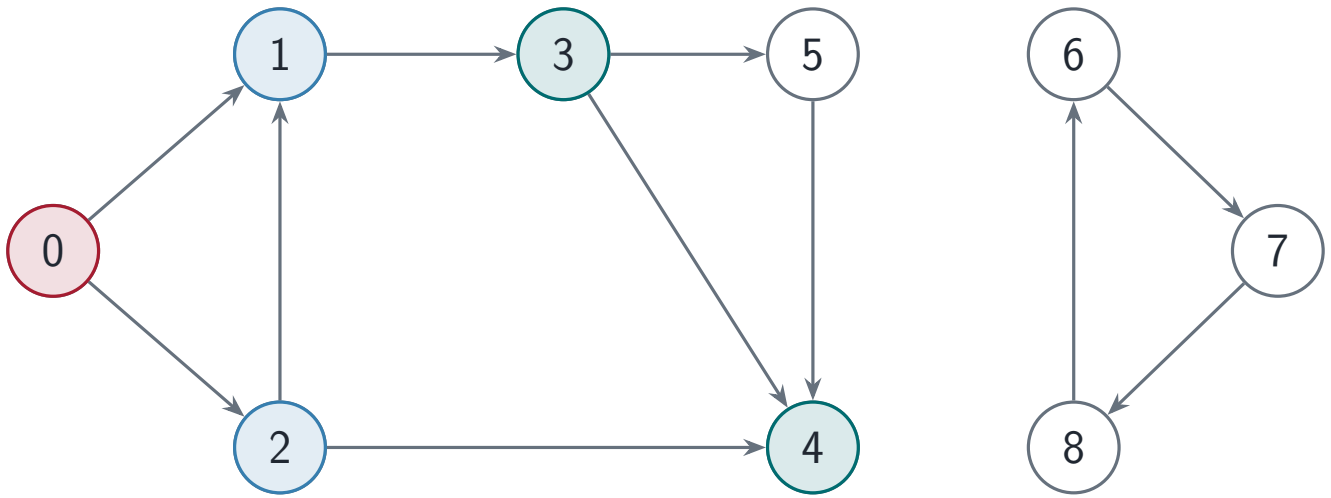
Example: $s = 0,\ t = 4$



$F_0 = \{0\}$ $F_1 = \{1, 2\}$

Finding the Paths

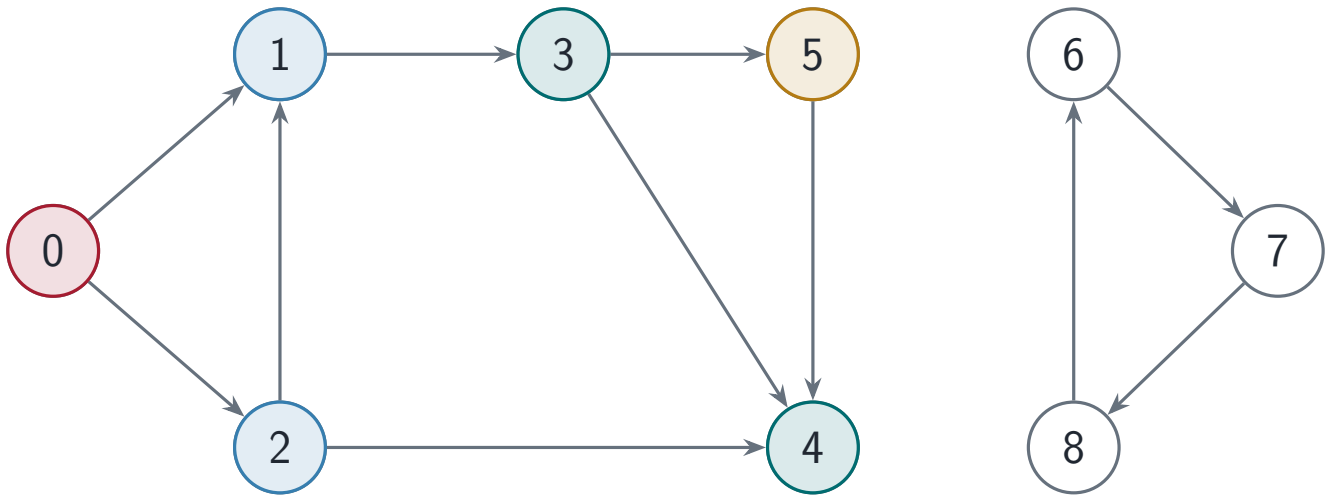
Example: $s = 0,\ t = 4$



$F_0 = \{0\}$ $F_1 = \{1, 2\}$ $F_2 = \{3, 4\}$

Finding the Paths

Example: $s = 0,\ t = 4$



$F_0 = \{0\}$ $F_1 = \{1, 2\}$ $F_2 = \{3, 4\}$ $F_3 = \{5\}$

Register allocation: coloring problem

For an undirected graph $G = (V, E)$, a proper *k-coloring* of G is a mapping $f : V \to [k]$ such that for all edges $\{u, v\} \in E$, we have $f(u) \neq f(v)$.

Register allocation: coloring problem

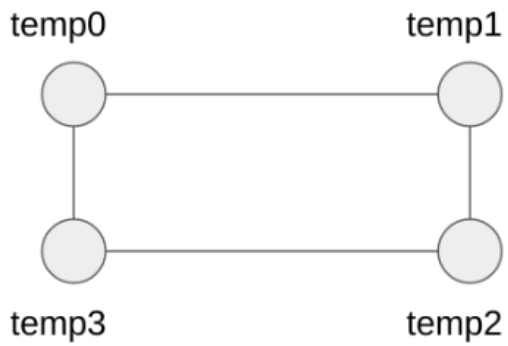
For an undirected graph $G = (V, E)$, a proper *k-coloring* of G is a mapping $f : V \to [k]$ such that for all edges $\{u, v\} \in E$, we have $f(u) \neq f(v)$.
The computational problem of finding a coloring is called the GraphColoring problem:

Register allocation: coloring problem

For an undirected graph $G = (V, E)$, a proper *k-coloring* of G is a mapping $f : V \to [k]$ such that for all edges $\{u, v\} \in E$, we have $f(u) \neq f(v)$.
The computational problem of finding a coloring is called the GraphColoring problem:
Input: A graph $G = (V, E)$ and a natural number k. **Output:** A k-coloring of G (if one exists)

Register allocation: coloring problem

For an undirected graph $G = (V, E)$, a proper *k-coloring* of G is a mapping $f : V \to [k]$ such that for all edges $\{u, v\} \in E$, we have $f(u) \neq f(v)$.
The computational problem of finding a coloring is called the GraphColoring problem:
Input: A graph $G = (V, E)$ and a natural number k. **Output:** A k-coloring of G (if one exists)
Q: Is this graph 2-colorable?



Q: What is the relation between colors and number of registers needed?

Greedy Coloring algorithm

Lets focus on a related problem: color a graph with as few colors as possible.

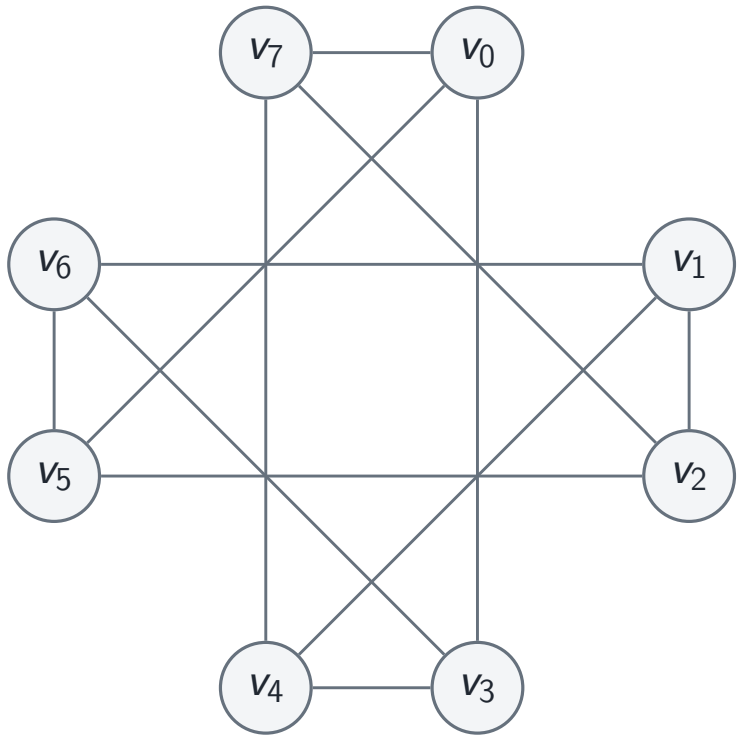
Greedy Coloring algorithm

Lets focus on a related problem: color a graph with as few colors as possible.

```
GreedyColoring(G):
Input      : A graph G = (V, E)
Output     : A coloring f of G using "few" colors
0 Select an ordering v_0, v_1, v_2, ..., v_{n-1} of V;
1 foreach i = 0 to n - 1 do
2 |  f(v_i) = min {c ∈ ℕ : c ≠ f(v_j)  ∀j < i s.t. {v_i, v_j} ∈ E}.
3 return f
```

Example

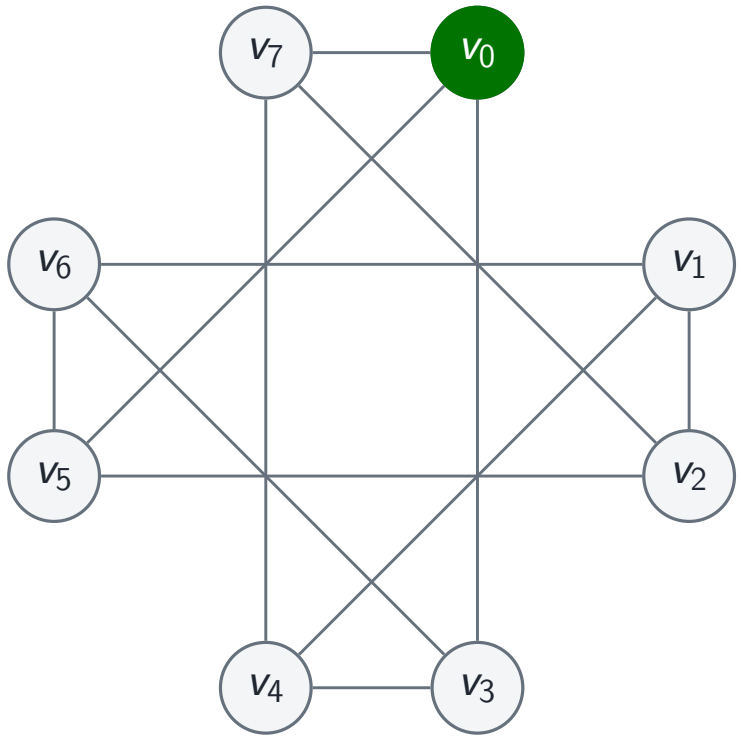
Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



Choose the smallest available color.
All vertices are initially uncolored.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



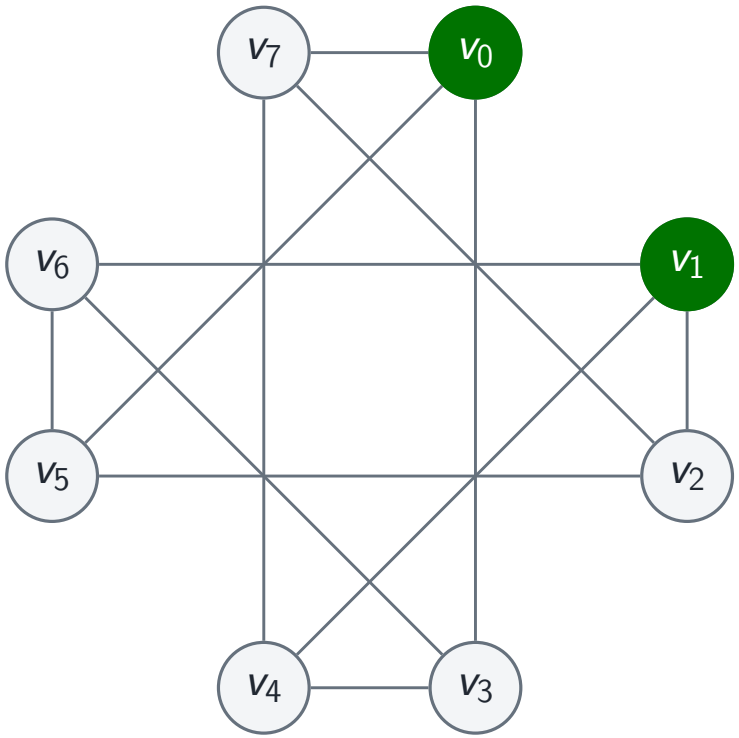
Choose the smallest available color.
Vertex v_0

Colors at already-colored neighbors: $\varnothing$.

$f(v_0) = 1$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



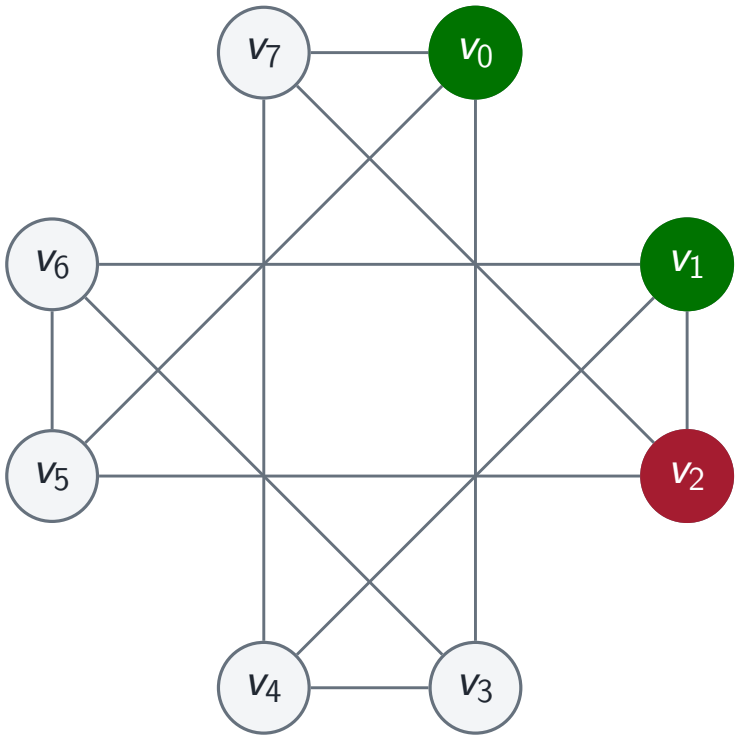
Choose the smallest available color.
Vertex v_1

Colors at already-colored neighbors: $\varnothing$.

$f(v_1) = 1$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



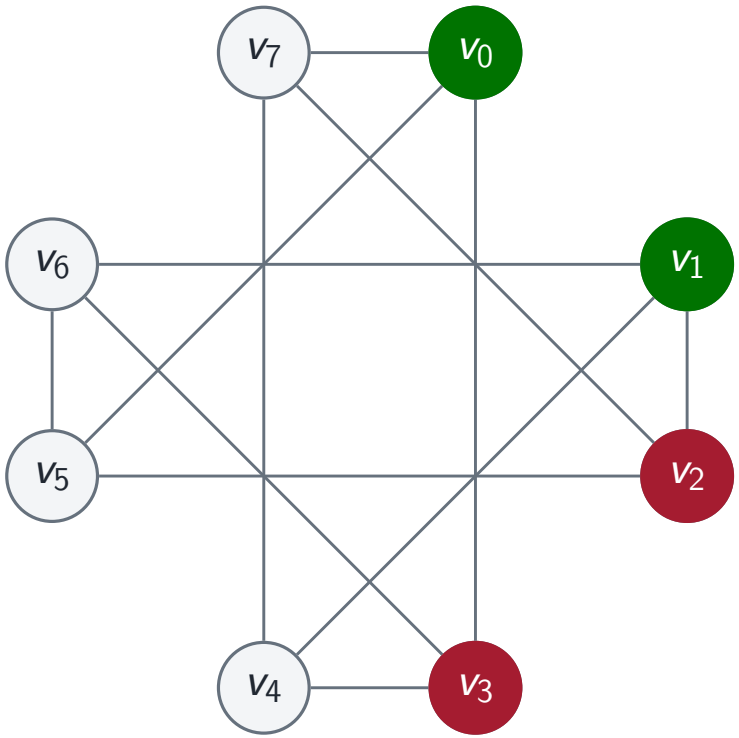
Choose the smallest available color.
Vertex v_2

Colors at already-colored neighbors: $\{1\}$.

$f(v_2) = 2$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



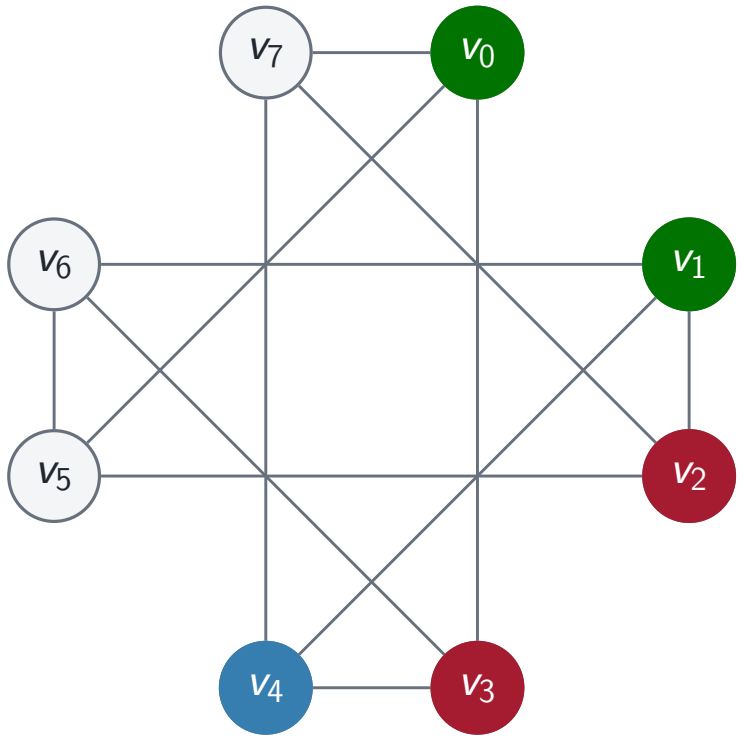
Choose the smallest available color.
Vertex v_3

Colors at already-colored neighbors: $\{1\}$.

$f(v_3) = 2$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



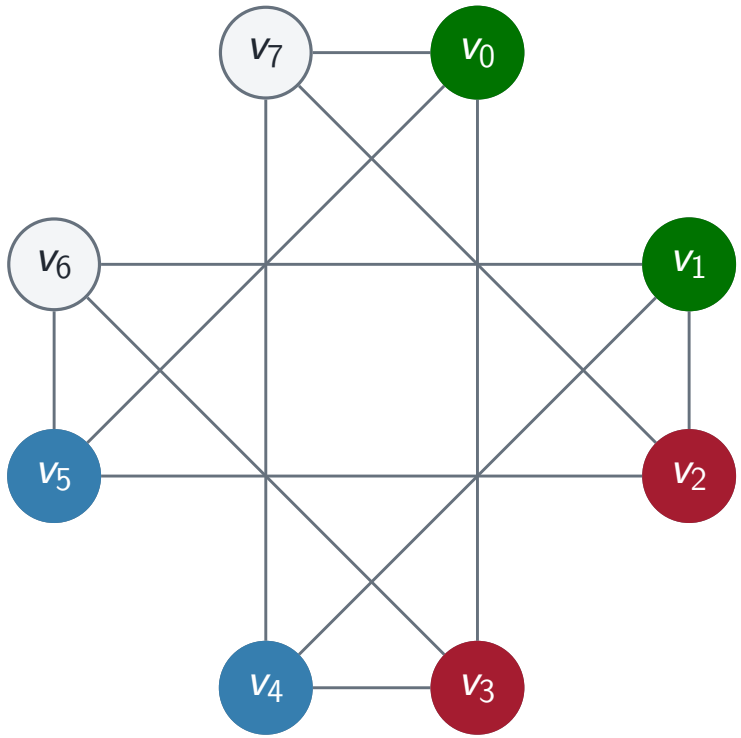
Choose the smallest available color.
Vertex v_4

Colors at already-colored neighbors: $\{1, 2\}$.

$f(v_4) = 3$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



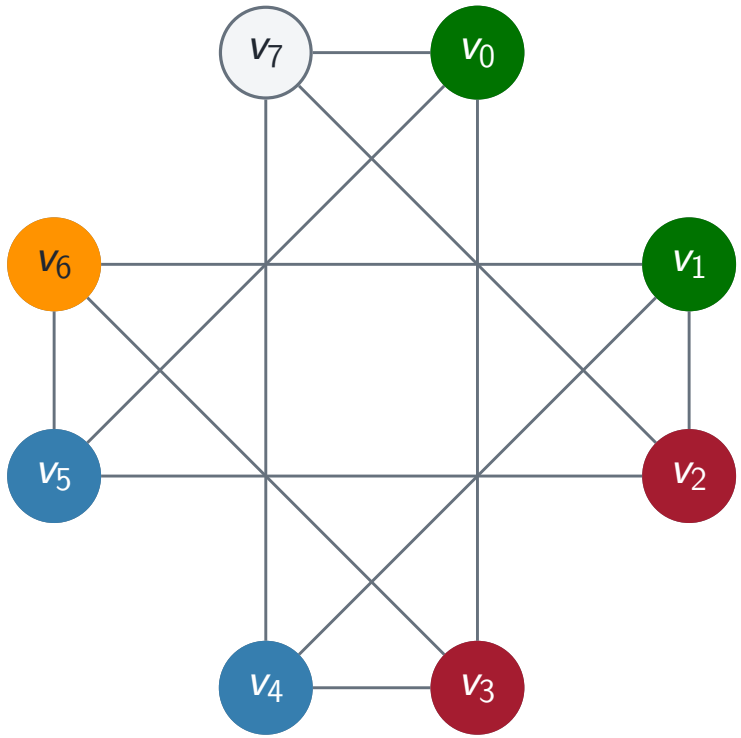
Choose the smallest available color.
Vertex v_5

Colors at already-colored neighbors: $\{1, 2\}$.

$f(v_5) = 3$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



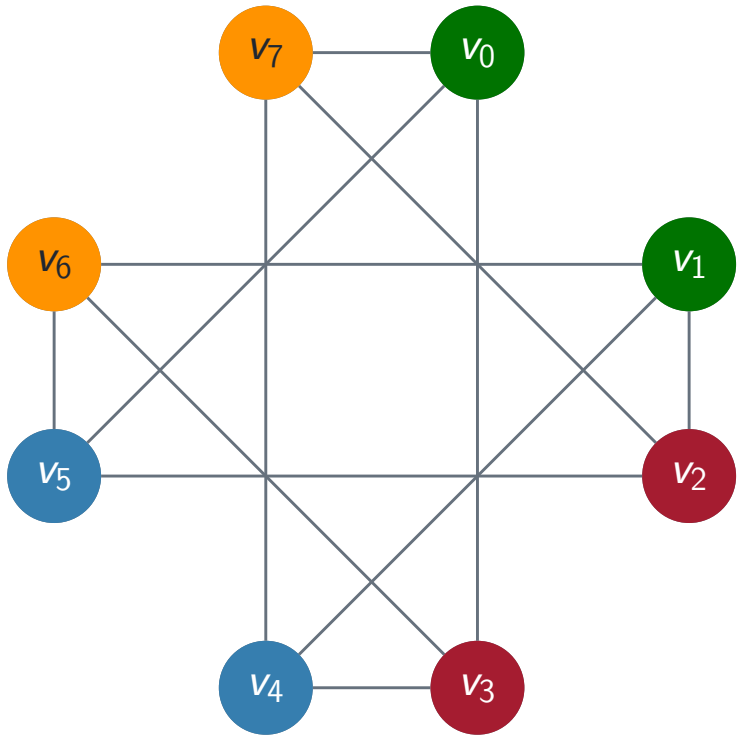
Choose the smallest available color.
Vertex v_6

Colors at already-colored neighbors: $\{1, 2, 3\}$.

$f(v_6) = 4$.

Example

Greedy coloring in clockwise order: $v_0, v_1, \ldots, v_7$**.**



Choose the smallest available color.
Vertex v_7

Colors at already-colored neighbors: $\{1, 2, 3\}$.

$f(v_7) = 4$.